Guillaume Trüeb
Résumé DMA TE1
15-04-2026

Communication sans fil

Le milieu air & Shannon

- Communication via **ondes électromagnétiques** (radio, micro-ondes)
- Capacité limitée par la formule de Shannon : $C = W \cdot \log_2\left(1 + \frac{S}{N}\right)$
- W = largeur de bande (limitée), $\frac{S}{N}$ = rapport signal/bruit
- Puissance reçue P_r décroît avec le **carré de la distance** et le **carré de la fréquence**

Modulation

Trois types principaux (variation d'un paramètre de l'onde porteuse) :
- **Amplitude** (AM), **Fréquence** (FM), **Phase** (PM)
- Critère de Nyquist : $f_m \leq 2f_p$ pour reconstruire le signal

Formule de Friis

La puissance reçue entre deux antennes : $P_r = P_t \cdot G_t \cdot G_r \cdot \left(\frac{\lambda}{4\pi d}\right)^2$
- P_t = puissance émise, G_t/G_r = gains antennes émettrice/réceptrice
- λ = longueur d'onde, d = distance

→ Doubler la distance → puissance divisée par **4**. Doubler la fréquence → puissance divisée par **4**.

Puissances d'émission typiques : LoRa **25 mW**, Wi-Fi **50 mW**, Mobile **1 W**

Augmenter la capacité du milieu air

Plusieurs leviers, tous avec des **contraintes** :
- **Fréquences plus élevées** → affaiblissement ∝ f² → portée réduite
- **Moins de bande** → GSM transmet 13 kbit/s un signal qui en demande 64 kbit/s
- **Plus de cellules** (picocells, femtocells) → coûteux, emplacements à trouver
- **Améliorer S/N** → difficile de réduire le bruit ; augmenter la puissance = limites légales, électrosmog, interférences
- **Meilleures techniques de transmission** → R&D intensive, nécessite remplacement hardware
- **MIMO** : plusieurs antennes Tx et Rx simultanément → multiplication du débit sans bande sup.
- **QAM** : moduler amplitude + phase ensemble → plusieurs bits par symbole (64-QAM = 6 bits/symbole)

Réseaux cellulaires

- Fréquences limitées → réutilisation par clusters (ex. taille 7)
- Types de cellules :

Type	Rayon	Puissance	Users
Macrocell	>10 km	40 W	2000
Microcell	1 km	10 W	200
Picocell	100 m	1 W	32
Femtocell	20 m	0.1 W	5

Évolution des générations mobiles

Gen	Année	Débit	Latence	Remarques
2G/GSM	1992	kbit/s	< 1000 ms	1er réseau numérique, voix
2.5G/GPRS	2000	171 kbit/s		Paquets, WAP
2.75G/EDGE	2003	473 kbit/s		«Poor man's UMTS»
3G/UMTS	2003	Mbit/s	< 500 ms	Données dès la conception
3.5G/HSPA+	2008	42 Mbit/s		Essor Internet mobile
4G/LTE	2010	150 Mbit/s	< 100 ms	Paquets uniquement, VoLTE
4.5G/LTE-A	2013	1′200 Mbit/s		Agrégation de porteuses
5G	2019	Gbit/s	< 5 ms	IoT, beamforming
6G	2030?	Tbit/s	0.1 ms	IA, IoT massif

4G / LTE — points importants :
- Réseau **100% paquets** → incompatible avec la voix traditionnelle → rebascule en 3G/2G pour les appels (10 s de délai)
- **VoLTE** (Voice over LTE) : ajout ultérieur permettant la voix sur 4G
- GSM arrêté en Suisse : **Swisscom & Salt début 2022**, **Sunrise début 2023**

5G — Bandes de fréquences & Beamforming

- **700 MHz – 1.8 GHz** : téléphonie intérieur, IoT
- **3.5 – 3.8 GHz** : téléphonie extérieur, 5G standard
- **26 – 28 GHz** : ultra haut débit, déploiement lent en Europe (pas encore attribué en Suisse)
- **Beamforming** : émission en faisceau étroit via array antenna → meilleur S/N, moins d'interférences

Wi-Fi (802.11)

- Protocole d'**accès réseau**, pas un réseau (pas d'itinérance native)
- Bandes **ISM** (libre) : 2.4 GHz, 5 GHz, 6 GHz
- **2.4 GHz** : 13 canaux (CH) dont seulement **3 non-chevauchants : CH1, CH6, CH11** → essentiels pour déploiement multi-AP sans interférence. Partagé avec Bluetooth et micro-ondes.

Norme	Année	Débit max
Wi-Fi 4 (802.11n)	2009	450 Mbit/s
Wi-Fi 5 (802.11ac)	2014	1′000 Mbit/s
Wi-Fi 6 (802.11ax)	2019	2′400 Mbit/s
Wi-Fi 6E	2021	4′800 Mbit/s
Wi-Fi 7 (802.11be)	2024	30′000 Mbit/s

LoRaWAN

- IoT / M2M, très bas débit (0.3-50 kbit/s), portée 10 km
- Batterie 10 ans, payload 51 bytes, duty-cycle 0.1-1%
- Réseau communautaire **The Things Network** (TTN), Swisscom a son propre réseau

Satellite

- iPhone 14+ : connectivité satellite pour **messages d'urgence** courts (≠ Starlink haut débit)
- Android 15 : service de messagerie satellite
- SpaceX : LTE par satellite (SMS en premier)

Li-Fi & uXDT

- **Li-Fi** : communication par **lumière visible** (LED) — débit théorique > 10 Gbps, portée < 10m, nécessite ligne de vue directe. Usage : zones sensibles (avion, hôpital) où le RF est interdit.
- **uXDT** (ultrasound cross-device tracking) : balises **ultrasonores inaudibles** émises par TV/écrans → microphone du smartphone capture le signal → tracking publicitaire cross-device sans réseau.
  - ⚠️ Vie privée : difficile à bloquer, transparent pour l'utilisateur

Contraintes applicatives

- Émission = consommation batterie → favoriser protocoles légers
- Connexion intermittente → prévoir **cache** et **look-ahead**
- Protocoles HTTP synchrones → masquer par couche asynchrone
- Dissymétrie uplink/downlink → souvent inversée dans les apps modernes

Électrosmog & santé

- Ondes mobiles = **non-ionisantes** (pas de rupture de liaisons moléculaires)
- **ICNIRP** — deux effets selon fréquence :
  - **< 10 MHz** : stimulation nerveuse et musculaire (effet électrique direct)
  - **> 100 kHz** : effet **thermique** (absorption et échauffement des tissus)
  - Zone de chevauchement 100 kHz–10 MHz : les deux effets coexistent
- Norme ICNIRP : 0.08 W/kg pour la population générale
- DAS max autorisé sur mobile : **2 W/kg**
- Aucun consensus scientifique sur des effets cancérigènes prouvés
- La Suisse applique des limites **7-10x plus strictes** que l'UE/ICNIRP (4-6 V/m vs 42-61 V/m)
- Luxembourg : 3 V/m, Bruxelles : 6 V/m, Paris : 5 V/m
- **DAS & ANFR** : mesures officielles réalisées en laboratoire dans des conditions standardisées qui ne reflètent pas toujours l'usage réel (position, distance du corps, mode réseau). Des associations critiquent la méthodologie.

Protocoles mobiles

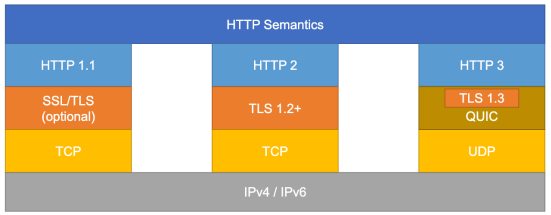
Le protocol HTTP (basé sur TCP/IP) s'est imposé comme protocol principal pour le web mais n'est pas forcément adapté au mobile.
- `XML-RPC`: non standardisé, mais concis et intéréssanst pour une solution ad-hoc
- `SOAP`: extrêmement verbeux et redondant, à déconseiller pour du mobile
- `REST`: très répandu pour les applications web
- `GraphQL`: développé par Facebook, relativement récent

Souvent utilisé avec des formats de données légers comme JSON ou XML.

TCP/IP a été conçu pour des réseaux stables et filaires, pas pour des connexions mobiles intermittentes et à haute latence.

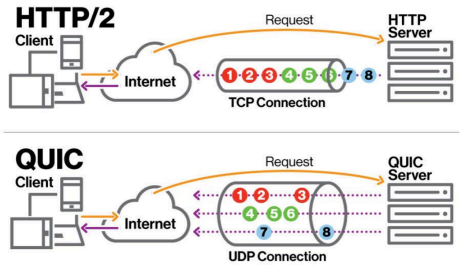
HTTP

- HTTP/1.1 figé pendant 20 ans
- **HTTP/2 (2015)** : multiplexage + compression des headers
- **HTTP/3 (2022)** : remplace TCP par **QUIC** (basé sur UDP)



HTTP/3 & QUIC

- TLS 1.3 **intégré** au transport → handshake plus rapide
- Requêtes **en parallèle** → pas de blocage si paquet perdu
- **Multipath** → Wi-Fi + LTE simultanément
- Fallback TCP si UDP bloqué par NAT
- Adoption rapide : 20% du trafic EU en 2021 (encore en brouillon)



Limites de TCP/IP sur mobile

- Overhead > payload sur petits messages
- Slow start inadapté à la latence mobile
- Handshake coûteux en temps
- Changement d'IP fréquent → connexion coupée

Particularités applicatives

- HTTP est **synchrone** (requête → réponse) → bloque l'app, **mauvais sur mobile**
- **iOS** : modèle de délégation imposé, pas d'accès direct aux threads
- **Android** : flexible (coroutines, threads) mais complexe

Modèle Android : le **Main/UI Thread** gère l'interface. Toute opération réseau/BDD/capteur doit se faire dans un **thread séparé** (sinon ANR — App Not Responding). Solutions : coroutines Kotlin, WorkManager, Handlers.

À retenir : toujours masquer le caractère synchrone des communications à l'utilisateur via des mécanismes **asynchrones**.

Sérialisation

En utilisant du HTTP, il existe deux technologies principales pour sérialiser les données :

XML

- Verbeux mais partiellement auto-documenté
- Validable via DTD ou XML Schema

```
<?xml version="1.0" encoding="UTF-8"?>
<endereco>
  <cep>31270901</cep>
  <city>Belo Horizonte</city>
  <shops>
    <shop><name>Loja 1</name><distance>500m</distance></shop>
  </shops>
</endereco>
```

XML DTD

Le DTD (Document Type Definition) définit la structure autorisée d'un document XML (éléments, attributs, relations). Limitation : typage très limité, tout est textuel (#PCDATA)

```
<!ELEMENT endereco (cep, city, shops)>
<!ELEMENT shops (shop*)>
<!ELEMENT shop (name, distance)>
<!ATTLIST shop id CDATA #REQUIRED status (OK|NEW|ERROR) #REQUIRED>
```

XML Schema

- Remplace le DTD avec plus de flexibilité
- Typage précis des données (ex: distinguer une date d'une valeur numérique)
- Limitation par intervalles, indicateurs d'occurrences, support des namespaces

Parseurs XML

- **SAX** : lecture streaming ligne par ligne, faible mémoire → adapté au mobile
- **DOM** : chargement complet en mémoire sous forme d'arbre → simple mais lourd
  - ex: un fichier XLSX de 30 Mo peut prendre 10 Go en mémoire !
- Certaines librairies permettent le **marshalling** direct vers des objets

Outils XML complémentaires

- **XPath** : langage de requête pour naviguer dans un document XML (ex: `/endereco/shops/shop[1]/name`)
- **XSL/XSLT** : transformation XML → XML/HTML/texte. Utile pour adapter un format serveur sans modifier le code client.

JSON

- Compact et concis, validation via librairies externes
- Favorisé si JavaScript côté client/serveur ; défavorable pour PHP/JS côté XML

```
{
  "city": "Belo Horizonte",
  "shops": [
    {"name": "Loja 1", "distance": "500m"}
  ]
}
```

Limites de la sérialisation textuelle :

- `4294967295` (32 bits = 4 B) → `{"v":4294967295}` = 16 B → efficacité 25%
- `false` (1 bit) → `{"v":false}` = 11 B → efficacité 1%

→ Protobuf binaire beaucoup plus efficace pour les nombres

- **Statistique clé** : dans un échange REST JSON typique, **6% du payload** sont des données utiles, **94% sont du overhead** (headers HTTP, balises JSON, chiffrement TLS…) → Justifie Protobuf + DEFLATE pour applications mobiles gourmandes en données

Introspection & Réflexion

Permet la sérialisation automatique d'objets sans tout écrire manuellement.

- Java/Kotlin offrent des APIs d'introspection puissantes → bien supporté sur Android

```
implementation("org.jetbrains.kotlin:kotlin-reflect:x.y.z")
val c = Person::class
c.constructors  // tous les constructeurs
c.members       // toutes les propriétés et fonctions
```

Java POJO : constructeur vide + setters/getters. **Kotlin data class** : `primaryConstructor` avec tous les paramètres dans l'ordre → pas de constructeur vide à chercher.

Sérialisation (exercice 1) :

```
// Parcourir les propriétés publiques mutables
classe.members
  .filter { it.visibility == KVisibility.PUBLIC }
  .filterIsInstance<KMutableProperty<*>>()
  .forEach { prop ->
    val value = prop.getter.call(objet)
    when {
      prop.returnType.isSubtypeOf(String::class.starProjectedType) ->
json.accumulate(prop.name, value)
      prop.returnType.isSubtypeOf(Date::class.starProjectedType)   ->
json.accumulate(prop.name, ISO_DATE_FORMAT.format(value))
      // Int, Double, Float idem
    }
  }
// ISO_DATE_FORMAT = SimpleDateFormat("yyyy-MM-dd'T'HH:mm:ss.SSSXXX")
```

Désérialisation (exercice 1) :

```
val data = JSONObject(json)
val constructor = classe.primaryConstructor   // data class →
constructeur principal
val params = arrayListOf<Any>()
constructor?.parameters?.forEach { param ->
  val value = data.get(param.name)  // récupère par nom de paramètre
  when {
    param.type.isSubtypeOf(Date::class.starProjectedType) ->
params.add(ISO_DATE_FORMAT.parse(value as String))
    else -> params.add(value)        // String, Int, Double, Float :
directement typé
  }
}
return constructor.call(*params.toTypedArray())  // opérateur spread
*
```

Protobuf (Protocol Buffers)

Le protobuf est un format de sérialisation **binaire** développé par Google. **2.5x plus compact que JSON**, génère du code client/serveur à partir du schéma `.proto`.

```
message Person {
  string          user_name       = 1;
  int64           favorite_number = 2;
  repeated string interests       = 3;
}
```

- Outils officiels : Java/Kotlin, Python, C#, Go, JS, Dart…
- **gRPC** : framework RPC Google basé sur Protobuf (alternative à XML-RPC)

Services web et API

Il existe plusieurs familles de service web:

- `RPC`: Remote Procedure Call, permet d'invoquer des fonctions à distance
- `SOAP`: Simple Object Access Protocol, basé sur XML, très verbeux
- `RESTful`: REpresentational State Transfer, utilise HTTP et des ressources identifiées par des URI

XML-RPC

- RPC est un protocole réseau permettant de faire des appels de procédures sur un ordinateur distant.
- XML-RPC utilise XML pour encoder les appels et les réponses, et HTTP comme protocole de transport.
- `POST /my_rpc`

```
<?xml version="1.0"?>
<methodCall>
  <methodName>examples.getStateName</methodName>
  <params>
    <param>
      <value><i4>40</i4></value>
    </param>
  </params>
</methodCall>
```

- Réponse

```
<?xml version="1.0"?>
<methodResponse>
  <params>
    <param>
      <value><string>South Dakota</string></value>
    </param>
  </params>
</methodResponse>
```

Critique : XML-RPC est **4x plus verbeux** que du simple XML sans les balises RPC → même résultat avec `POST /examples/getStateName` et un payload `<id>40</id>`. Alternative JSON : **JSON-RPC**.

SOAP

- Le protocole SOAP est proche de XML-RPC mais plus complexe et relativement lourd (XML Schema, namespace, etc.).
- Avantage: permet de décrire les services et types de données offerts par un service SOAP via un fichier WSDL (Web Services Description Language), ce qui facilite la découverte et l'intégration des services.

```
<!-- Requête SOAP -->
<Envelope xmlns="http://schemas.xmlsoap.org/soap/envelope/">
  <Body><GetStudentDetailsRequest xmlns="http://in28minutes.com/
students">
    <id>1</id>
  </GetStudentDetailsRequest></Body>
</Envelope>
<!-- Réponse : même enveloppe + namespaces → très verbeux -->
```

RESTful

- REST (REpresentational State Transfer) est un style d'architecture pour les services web avec quelques contraintes:
- Séparation client/serveur
- Sans état

- Mise en cache
- Interface uniforme (chaque ressource a une URI unique)

Un service respectant ces contraintes est dit RESTful.
- Permet l'utilisation de `JSON` ou `XML`, mais JSON est plus courant.
- Opérations CRUD :
  - `GET /objects` / `GET /objects/23` → lire
  - `POST /objects` → créer
  - `PUT /objects/23` → remplacer, `PATCH` → modifier, `DELETE` → supprimer

Problème N+1
- `GET /author/345` retourne une liste d'URIs de livres → N requêtes supplémentaires
- Solution : agréger les données côté serveur, ou utiliser GraphQL

Multiplaxage
- Permet d'envoyer plusieurs requêtes en parallèle sur la même connexion TCP (HTTP/2) ou même de les faire coexister (HTTP/3 avec QUIC)
- Réduit la latence en évitant les blocages liés à l'attente de réponses
- Important pour les applications mobiles où la latence est un facteur critique

```
<?xml version="1.0" encoding="utf-8"?>
<apdu>
  <opdu className="ch.iict.dma.server.storeTemperature">
    <param name="lat" value="46.78" />
    <param name="lon" value="6.65" />
    <param name="temp" value="23.8" />
  </opdu>
  <opdu className="ch.iict.dma.server.docQuery">
    <param name="docId" value="345678" />
  </opdu>
  <opdu className="ch.iict.dma.server.messageAck">
    <param name="msgId" value="234" />
    <param name="ack" value="true" />
  </opdu>
</apdu>
```

GraphQL
- Développé par Facebook, permet au client de spécifier exactement les données dont il a besoin
- Réduit le surcoût de données inutiles (overfetching) et les requêtes multiples (underfetching)
- Plus complexe à mettre en place que REST, mais offre une grande flexibilité pour les clients ce qui est pratique pour les applications mobiles

Il permet de faire du requêtage (plus ou moins complexe) sur les données exposées par le serveur, et de n'obtenir que les champs nécessaires à l'affichage de l'interface utilisateur.

```
{ "query": "{findAllAuthors{id, books{id, title}}}" }
// → retourne uniquement les champs demandés, en suivant les
relations
```

Evolution du protocole
- Prévoir dès la v1 comment gérer les futures versions (v2, v3…)
- Sur mobile, difficile de synchroniser la mise à jour app + backend → certains utilisateurs restent sur d'anciennes versions
- **Parseurs stricts** (ex: Xstream) : refusent de désérialiser si le modèle a changé → oblige à garder l'historique de tous les modèles côté serveur → vite ingérable
- **Parseurs permissifs** (ex: Gson) : variables manquantes = `null` → plus flexible, mais le code doit gérer les cas null

Objets volumineux
- Sérialisation JSON peut vite atteindre plusieurs Mo
- Problématique sur connexion lente ou intermittente
- **Solution** : compresser les données (ex: algorithme DEFLATE)
  - Attention : POST compressé n'est pas standard HTTP → nécessite la maîtrise du client **et** du serveur
  - Fichiers déjà compressés (jpeg, mp3, png…) → inutile de recompresser
  - Base64 → rarement la bonne solution (augmente la taille de 33%)

Labo 1 — Protocoles de communication en pratique

Le labo 1 tester toutes les combinaisons de : **sérialisation** (JSON / XML+DTD / Protobuf) × **compression** (aucune / DEFLATE) × **chiffrement** (HTTP / HTTPS) × **type de réseau simulé** (CSD → 5G NR), et mesurait le temps de chaque requête.

Modèle de données Measure : `id`, `status` (NEW/OK/ERROR), `type` (TEMPERATURE/PRECIPITATION/HUMIDITY/PRESSURE), `value: Double`, `date: Calendar`

Conclusions clés :
- **XML + DTD** : verbeux mais validable côté serveur (structure garantie)
- **JSON** : compromis taille/lisibilité, pas de validation intégrée
- **Protobuf** : le plus compact (2.5× vs JSON), nécessite le `.proto` partagé
- **DEFLATE** : réduit le payload mais ajoute du CPU (surtout pertinent sur réseaux lents : GPRS/EDGE/UMTS)
- **HTTPS/TLS** : overhead du handshake non négligeable sur réseau lent, mais obligatoire
- La mesure de `elapsed = measureTimeMillis { … }` permet de comparer objectivement les approches

GraphQL (labo 1) : deux requêtes distinctes — liste de tous les auteurs (`findAllAuthors{id, name}`) puis livres d'un auteur sélectionné (`findBooksFromAuthor`) → illustre comment GraphQL évite l'overfetching vs REST.

Push (labo 1) : réception de messages FCM → stockés en base via Room (`MessagesDao`), affichés dans un Fragment via LiveData → architecture MVVM complète.

API Gateway & Serveur de mobilité

API Gateway
- Sur architecture **micro-services**, le client doit appeler plusieurs services → complexité, couplage, surface d'attaque
- **API Gateway** : point d'entrée unique, gère authentification, SSL, Firebase, agrégation, caching, logging
- Permet d'adapter la granularité des données aux besoins mobile (moins de requêtes, données agrégées)

Serveur de mobilité
- Permet de **cacher** des services existants non adaptés au mobile
- Gère : connecteurs, logique métier, contrôle d'accès, **synchronisation**, Firebase CM
- Problème de synchronisation : deux clients modifient la même ressource hors-ligne → **conflits à résoudre**

Polling & Notifications

Connexions actives
Quand le trafic serveur→client est important, on peut maintenir une connexion ouverte :
- **Sockets TCP/IP** : socket persistant, le serveur pousse les données
- **Websockets** (ex: Socket.io) : couche sur HTTP, bidirectionnel
- **HTTP/1.1 Long Polling** : client attend la réponse indéfiniment
- **HTTP/2 Server Push** : retiré de Chrome/Chromium
- Très énergivore → impossible en arrière-plan sur mobile (Doze, etc.)

Active Poll
- Le client interroge **périodiquement** le serveur pour savoir si une info est disponible
- Intéressant si les interrogations restent **rares**
- Exemple : vérification d'emails toutes les 15 min (IMAP/POP3)

Contraintes mobiles (Android)
Les OS mobiles mettent les apps en veille pour économiser l'énergie :
- **Doze** (API 23) : désactive réseau, Wi-Fi, synchronisation…
- **App Standby Buckets** (API 28) : priorise les apps selon l'usage
- **App Hibernation** (API 30) : mise en veille profonde
- Le smartphone ne se réveille que lors de **fenêtres de maintenance**
- Sur Android : active poll limité à **toutes les 15 min** maximum
- Solutions :
  - **Jetpack WorkManager** : synchroniser le poll avec ces fenêtres
  - **Foreground notification** : pour garder la main sur l'app

Lazy Poll
- Problème de l'active poll : beaucoup de requêtes inutiles (pas de nouvelles données)
- Solution : profiter des requêtes **existantes** pour transmettre une info en retour
  - Ex: lors de l'envoi d'un message, le serveur ajoute un header indiquant qu'une mise à jour est disponible
- Souvent combiné avec l'active poll pour garantir une réactivité minimum

Poll on Request
- Utiliser un **canal externe** pour notifier le client qu'une info est disponible
- Historique :
  - SMS : l'app écoutait les SMS, exécutait la commande et supprimait le SMS
  - Blackberry : push email via extension du protocole téléphonique
- Aujourd'hui : accès SMS/téléphonie très limité pour apps tierces
- Solutions actuelles :
  - **Firebase Cloud Messaging (FCM)** → Android (Google)
  - **Apple Push Notifications (APN)** → iOS

Firebase Cloud Messaging
- Service **gratuit** de Google
- Permet d'envoyer notifications/messages push à un user, un groupe ou tous
- Payload très limité et non sécurisé → utilisé uniquement pour **notifier** que des données sont à récupérer sur le serveur
- Nécessite **Google Play Services** → pas disponible sur tous les appareils
- ⚠️ **Vie privée** : même sans payload sensible, savoir qu'un message a été échangé est une information exploitable — dépendance à un tiers (Google/Apple)

Techniques complémentaires
- **Multiplexage** : regrouper plusieurs messages en une seule requête
- **Mise en cache & look-ahead** : anticiper les données nécessaires
- **Envoi différé** : profiter d'une connexion existante pour envoyer des données secondaires

Géolocalisation et positionnement en intérieur

Principes de base
- Localisation = se référer à des **points de référence** (naturels, artificiels, satellites…)
- Deux techniques :
  - **Triangulation** : utilisation des angles
  - **Trilatération** : utilisation des distances

Coordonnées géographiques
- La Terre = ellipsoïde → nécessite une **projection** pour représentation plane
- **Suisse** : système MN03 (1903) → MN95 (1995, officiel depuis 2016)
  - Axes x/y en mètres, y toujours > x sur territoire suisse
  - HEIG-VD : 2′540′503.0, 1′181′241.4
- **WGS 84** : standard mondial utilisé par GPS
  - Basé sur méridien de Greenwich + équateur
  - HEIG-VD : 46°46′46.286″N 6°39′33.587″E

GNSS / GPS

- GPS : 32 satellites opérationnels, altitude 20′184 km, période 12h
- Principe : satellites émettent timestamp → récepteur calcule distances → trilatération
- **A-GPS** : téléchargement des éphémérides via Internet → premier fix plus rapide
- Précision civile : 3-5m (signal volontairement dégradé jusqu'en 2000 : 100m)
- **Constellations & référentiels** :

Constellation	Pays	Référentiel
GPS	USA	WGS 84
GLONASS	Russie	PZ-90
Galileo	EU	GTRF
BeiDou	Chine	CTRF

- Smartphones actuels peuvent utiliser **toutes les constellations**
- **Effets relativistes** : les horloges atomiques des satellites sont affectées (relativité générale → plus rapides de +45 µs/jour, restreinte → plus lentes de −7 µs/jour) → correction logicielle obligatoire, sinon 10 km d'erreur/jour

Améliorer la précision

- **DGPS** : stations fixes corrigent les erreurs → précision métrique/décimétrique
- **RTK** : mesure de phase de l'onde porteuse → précision centimétrique
  - Usage: aviation, drones, topographie (pas directement sur smartphone)

Géolocalisation sans GNSS

- GNSS limité : ne fonctionne pas en intérieur, consomme la batterie
- Alternatives :
  - **Wi-Fi / Réseaux mobiles** : fingerprint DB propriétaire (Google/Apple)
    - Précision 50m (Wi-Fi), moins pour mobile
    - Partage de position avec service tiers
  - **Balises BLE**, **Balises UWB**

Exemple réel - Manifestation à Payerne

- Téléphones allumés à Payerne mais géolocalisés à Aarau ou Grisons
- Raison : la DB de localisation mobile utilise la position des antennes, pas la position réelle → erreurs possibles selon la configuration du réseau

Mise en œuvre Android

- **LocationManager** (SDK natif) : GPS_PROVIDER, NETWORK_PROVIDER, FUSED_PROVIDER (API 31+)
- **FusedLocationProviderClient** (Google Play Services, recommandé) :
  - Mutualise la localisation entre apps, gère les sources automatiquement
  - Mode **passif** : `lastLocation` → retourne la dernière position connue par n'importe quelle app
  - Mode **actif** : `requestLocationUpdates` avec `LocationRequest`

```
// Exercice 3 — FusedLocationProviderClient
val locationRequest = LocationRequest.Builder(interval) // ms
  .setPriority(QUALITY_HIGH_ACCURACY)     // 1s  → GPS
  // QUALITY_BALANCED_POWER_ACCURACY      // 10s → Wi-Fi/réseau
  // QUALITY_LOW_POWER                    // 60s → réseau uniquement
  .build()
fusedLocationProviderClient.requestLocationUpdates(locationRequest,
callback, Looper.myLooper())
// Désabonnement obligatoire (ex: onPause)
fusedLocationProviderClient.removeLocationUpdates(callback)
```

Problème LocationManager avec plusieurs sources (exercice 3) : GPS et NETWORK émettent de façon non coordonnée → il faut filtrer. Critères `isBetterLocation` :

- Position plus récente de >30 s → toujours meilleure
- Position plus précise (delta accuracy < 200 m) → meilleure
- Même source + plus récente + pas nettement moins précise → meilleure
- Permissions requises :
  - `ACCESS_COARSE_LOCATION` : 2km
  - `ACCESS_FINE_LOCATION` : haute précision
  - `ACCESS_BACKGROUND_LOCATION` : arrière-plan

Géocodage inverse

- Convertir coordonnées brutes → adresse/point d'intérêt
- Repose sur DB en ligne → pas toujours dispo hors réseau
- Service souvent payant/limité → ne requêter que si déplacement significatif
- **Appel bloquant** → doit s'exécuter dans un thread/coroutine (IO dispatcher) :

```
// Exercice 3 — géocodage inverse dans une coroutine
lifecycleScope.launch {
  withContext(Dispatchers.IO) {
    val addresses = Geocoder(context).getFromLocation(lat, lon, 10)
    // deprecated API 33+ → préférer callback, ou coroutine IO comme
ici
    withContext(Dispatchers.Main) { /* afficher le résultat */ }
  }
}
```

Geofencing

- Surveiller entrée/sortie d'une zone géographique **circulaire**
- Déléguer au **système** (iOS/Android) la surveillance → économise la batterie
- Bonnes pratiques :
  - Rayon minimum **100-150m** (surtout en intérieur, précision Wi-Fi 50m)
  - Réactivité de 5 min pour économiser l'énergie
  - Transparent avec l'utilisateur pour la permission background
  - Si Wi-Fi désactivé → détection moins fiable

```
// Permissions : ACCESS_FINE_LOCATION + ACCESS_BACKGROUND_LOCATION
geofenceList.add(Geofence.Builder()
  .setRequestId(key)
  .setCircularRegion(latitude, longitude, radius)
  .setExpirationDuration(expirationMs)
  .setNotificationResponsiveness(responsivenessMs) // ex: 5 * 60 *
1000
  .setTransitionTypes(
    Geofence.GEOFENCE_TRANSITION_ENTER or
Geofence.GEOFENCE_TRANSITION_EXIT)
  .build())
// Réponse via BroadcastReceiver (pas une Activity : impossible en
background)
```

Positionnement en intérieur

Balises BLE - iBeacons

- Introduit par Apple en 2013, basé sur **BLE** (portée 100m)
- Fonctionnement : balise broadcast en permanence (pas de connexion) → app détecte la balise → déduit sa position
- Format iBeacon : 31 bytes → UUID (16b) + Majeur (2b) + Mineur (2b)
  - Ex: majeur = étage, mineur = salle
- **Zones de proximité** (définies par Apple, valeurs indicatives) :
  - **Immediate** : < 0.5 m
  - **Near** : 0.5–3 m
  - **Far** : > 3 m
  - **Unknown** : RSSI trop faible pour estimer
- Limites :
  - Estimation de distance **peu fiable** (basée sur RSSI)
  - Toutes les annonces ne sont pas forcément reçues → il faut **lisser**
- Sécurité :
  - **Spoofing** : cloner une balise (même UUID/major/minor)
  - **Piggybacking** : utiliser les IDs d'une balise tierce
- Alternatives : **AltBeacon** (open source), **Eddystone** (Google, arrêté en 2021)

Labo 2 — iBeacons en pratique

Librairie : `org.altbeacon:android-beacon-library`

Parser iBeacon (format Apple) :

```
beaconManager.beaconParsers.add(
BeaconParser().setBeaconLayout("m:2-3=0215,i:4-19,i:20-21,i:22-23,p:24-24")
)
// id1=UUID, id2=major, id3=minor — callback :
didRangeBeaconsInRegion() ~1/s
```

Identification stable : clé `"$major-$minor-$uuid"` → `stableId` interne persistant entre les annonces pour éviter de recréer les objets.

Lissage des annonces : la librairie fait déjà une **moyenne sur 20s** (filtre les 10% de valeurs extrêmes du RSSI). Mais une balise peut quand même disparaître ponctuellement → cache horodaté avec timeout :

```
// Retirer uniquement les balises non vues depuis > 5s
beaconCache.entries.removeIf { now - it.value.second > 5000L }
_closestBeacon.postValue(active.minByOrNull { it.distance })
```

Calcul de distance : régression de puissance basée sur RSSI et TxPower :

- **TxPower** : puissance calibrée à 1 m, en dBm — valeur fixe stockée dans la balise
- **RSSI** : puissance mesurée localement par le chipset — non transmise dans le paquet BLE
- Formule : $d = A \cdot \left(\frac{r}{t}\right)^B + C$ (constantes empiriques par modèle d'appareil, défaut = Nexus 5)
- **Fiabilité réelle** : à 1 m → 0.5–2 m d'erreur ; à 20 m réels → 10–40 m estimés
- Tri par distance estimée croissante : plus représentatif qu'un tri brut par RSSI seul

Détermination de position : `closestBeacon = active.minByOrNull { it.distance }` → nom du lieu via `Map<Pair<major, minor>, String>`.
Méthodes pour améliorer :

- **Trilatération** : 3 balises visibles simultanément → position plus précise
- **Filtre de Kalman** : lissage temporel du RSSI/distance
- **Fusion de capteurs** : accéléromètre + gyroscope pour combler le délai de 20 s
- **Historique** : ne changer de position que si la balise dominante est stable sur plusieurs annonces

Monitoring en arrière-plan (3 stratégies de la librairie) :

Stratégie	Fréquence BG	Contrainte
JobScheduler (défaut)	15 min	Insuffisant pour temps réel
IntentScanStrategy	1 Hz si balises visibles	Consommation batterie élevée
ForegroundServiceStrategy	Continu	Notification persistante obligatoire Android 8+ : 10 min sans foreground Android 12+ : restrictions démarrage BG

Cas d'usage adaptés : navigation par salle dans musées/hôpitaux/aéroports ; notifications contextuelles en commerce (granularité quelques mètres suffisante).

Limitations : précision insuffisante pour sub-métrique ; coût/maintenance infrastructure (batteries à remplacer) ; signal fortement perturbé par corps humains, structures métalliques et autres appareils BT.

Balises UWB - AirTags

- Apple AirTags (2021) : BLE (position grossière) + UWB (position précise)
- Fonctionnement :
  1. Tag diffuse avec clé publique
  2. iPhones à proximité captent et remontent la position à Apple
  3. Propriétaire récupère la localisation
- UWB : étalement de spectre, impulsions courtes, robuste aux interférences
- APIs : iOS (publique), Android Samsung (privée), Android Jetpack beta (mars 2026)
- Alternatives :
  - **Samsung SmartTag+** : BLE ou BLE+UWB, Samsung uniquement
  - **Oppo Zero-Power Tag** : sans batterie, énergie via champ EM (4G/5G), backscattering

BLE 5.1

- Mesure des **angles** de départ/arrivée des signaux BLE
- Fonctionnalité optionnelle, pas encore répandue sur smartphones

Wi-Fi RTT (802.11mc FTM)

- Disponible depuis **Android 9**, peu d'AP compatibles
- Distance basée sur **temps de propagation** → trilatération avec 3+ AP
- Précision **< 1m** en intérieur (moyenne sur plusieurs mesures)

Centrale Inertielle (IMU)

- Gyromètres + Accéléromètres + Magnétomètre (optionnel)
- Intégration des mouvements depuis un point connu → position actuelle
- Limitations : **drift** (erreurs cumulatives) + capteurs smartphones insuffisants

Protocoles de proximité

USB

- Présent sur tous les mobiles récents, fiable pour gros volumes de données
- Smartphone peut être **périphérique** (ex: debug Android Studio) ou **hôte** (brancher un périphérique)

USB-C

- Connecteur réversible 24 pins : données + alimentation
- **Power Delivery** :
  - Base : 5V x 0.5A = 2.5W
  - Max : 48V x 5A = 240W
- Standards de débit :
  - USB 2.0 : 480 Mbps (2000)
  - USB 3.2 Gen 2 : 10 Gbps (2013)
  - USB 4 v2.0 : 80/120 Gbps (2022)
- **Thunderbolt** = version certifiée d'USB (TB3 → base d'USB4)
- Aucun smartphone n'embarque USB4/Thunderbolt en 2026
- ⚠️ **Les câbles USB-C se ressemblent tous physiquement** mais peuvent avoir des capacités très différentes (USB 2.0 vs USB4, 60W vs 240W) → toujours vérifier les spécifications du câble

Android

- Périphériques supportés nativement : stockage, clavier, souris, Ethernet…
- SDK permet d'intégrer un **pilote USB custom** pour périphériques spécifiques

iOS

- Accessoires non natifs → **programme MFi** (certification Apple obligatoire)
- Coût 4$ par accessoire vendu, NDA strict
- Lightning : débit USB 2.0 → iPhone 15 : transition vers USB 3.2 Gen 2
- Alternatives : port jack (supprimé en 2016), Ethernet nativement supporté

Codes-barres

1D

- Données encodées dans l'épaisseur/espacement des barres
- Formats : Code-128, EAN-13, UPC-A, Code-39…
- Lecture par **laser**, densité d'information faible → surtout pour identifiants

2D

- Formats : **QR Code**, Data Matrix, Aztec, PDF417…
- Lecture par **analyse photo**
- QR Code : correction d'erreurs intégrée (4 niveaux : L=7%, M=15%, Q=25%, H=30%)
- Capacité max : 7'089 chiffres / 4'296 alphanum / 2'953 ISO-8859-1
- Tailles : Version 1 (21x21) → Version 40 (177x177)

Types de contenu QR

- URL, vCard, événement, géolocalisation, Wi-Fi, email, téléphone…
- **Texte libre** : JSON, données custom (ex: certificat COVID en Base45)
- **GS1** : remplacera les EAN d'ici 2027 (identifiant + URI produit + métadonnées)
- **Dynamiques** : contenu modifiable, redirection selon OS, statistiques d'usage
  - ⚠️ Confiance au fournisseur requise (disponibilité, vie privée)

Lecture sur Android

- **zxing** : librairie Java, licence Apache, mode maintenance
- **ML Kit** : librairie Google ML, modèles entraînés, Terms of Service Google

Exercice 4 — CameraX + MLKit

Problème avec les QR codes grandes versions (v25, v40) : résolution par défaut de CameraX = **640×480** → insuffisant. Règle : min 2 px par module QR, en pratique 4-8 px. Solution → forcer la résolution à **5 MP** :

```
// Exercice 4 — résolution CameraX pour grands QR codes
val resolutionSelector = ResolutionSelector.Builder()
  .setResolutionStrategy(ResolutionStrategy(
    Size(2592, 1920),  // 5MP
    ResolutionStrategy.FALLBACK_RULE_CLOSEST_HIGHER_THEN_LOWER))
  .build()

val barcodesUseCase = ImageAnalysis.Builder()
  .setResolutionSelector(resolutionSelector)
  .setBackpressureStrategy(ImageAnalysis.STRATEGY_KEEP_ONLY_LATEST) //
une image à la fois
  .build().apply {
    setAnalyzer(Executors.newSingleThreadExecutor()) { proxy ->
processImageProxy(proxy) }
  }

// Limiter aux formats utiles (meilleure performance)
val barcodeClient = BarcodeScanning.getClient(
BarcodeScannerOptions.Builder().setBarcodeFormats(Barcode.FORMAT_QR_CODE).build()
)
// Fermer le proxy après traitement !
imageProxy.image?.close(); imageProxy.close()
```

Conclusion : QR v40 (177×177) utilisable techniquement mais pas adapté au grand public (difficile à scanner en conditions réelles).

NFC

- Classe particulière de **RFID**, lecture à max 3-4 cm
- Tag peut être **passif** (alimenté par champ EM du lecteur)
- Communication **bidirectionnelle**

Modes de fonctionnement

- **HCE** (émulation de carte) : terminal mobile agit comme carte sans contact (paiement)
- **Lecture/Écriture** : lire/écrire des tags passifs
- **Peer-to-peer** : échange direct entre 2 appareils NFC

Technologies & Tags

- NFC-A/B/F/V, MifareClassic, MifareUltralight
- Tags courants : NTAG213 (144b), NTAG216 (888b), SLIX (112b)

NDEF

- Format standardisé de messages lisibles sur la plupart des tags
- Types Well-Known : **URI**, **TEXT**, **SMARTPOSTER**
- Travailler au plus haut niveau d'abstraction recommandé

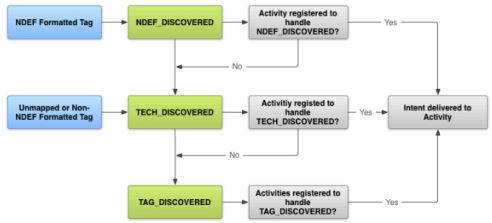
Android vs iOS

- **Android** : SDK très complet, tous niveaux d'abstraction, quasi tous modèles équipés
- **iOS** :
  - HW depuis 2014, API tierce depuis 2017 (lecture NDEF uniquement)
  - 2019 : écriture NDEF, 2022 : Commission EU accuse Apple d'abus de position dominante
  - iOS 18.1 : API étendue pour paiements/billetterie, pays et devs sélectionnés uniquement

Utilisation Android

```
// Permission requise dans le Manifest
<uses-permission android:name="android.permission.NFC" />

// 3 niveaux d'abstraction (du plus spécifique au plus général)
ACTION_NDEF_DISCOVERED   // tag NDEF → niveau recommandé
ACTION_TECH_DISCOVERED   // technologie spécifique (NFC-A, Mifare,
etc.)
ACTION_TAG_DISCOVERED    // n'importe quel tag NFC
```



Exercice 5 — ForegroundDispatch (lire quand l'activité est active)

```
// onResume → activer
override fun onResume() {
  super.onResume()
  val intent = Intent(applicationContext, this::class.java).apply {
    flags = Intent.FLAG_ACTIVITY_SINGLE_TOP  // évite plusieurs
instances
  }
  val pendingIntent = PendingIntent.getActivity(
  applicationContext, 0, intent, PendingIntent.FLAG_MUTABLE) //
FLAG_MUTABLE requis API 31+
  val filters =
arrayOf(IntentFilter(NfcAdapter.ACTION_NDEF_DISCOVERED))
  val techLists = arrayOf(arrayOf("android.nfc.tech.Ndef"))
  nfcAdapter.enableForegroundDispatch(this, pendingIntent, filters,
techLists)
}

// onPause → désactiver
override fun onPause() { super.onPause();
nfcAdapter.disableForegroundDispatch(this) }

// Réception du tag via onNewIntent
override fun onNewIntent(intent: Intent) {
  super.onNewIntent(intent)
  val rawMessages =
intent.getParcelableArrayExtra(NfcAdapter.EXTRA_NDEF_MESSAGES)
  rawMessages?.forEach { msg ->
    (msg as NdefMessage).records.forEach { record -> /* parser */ }
  }
}
```

Format des enregistrements NDEF Well-Known (exercice 5)

RTD_TEXT	RTD_URI
Byte 0 : flags (bit7=encodage, bits5-0=longueur lang) Puis code langue (ASCII) Puis texte (UTF-8 ou UTF-16)	Byte 0 : préfixe URI 0x01=http://www. 0x02=https://www. 0x03=http:// 0x04=https:// Puis reste de l'URI (UTF-8)

```
// Décodage RTD_TEXT
val flags = payload[0].toInt()
val textEncoding = if ((flags and 0x80) == 0) Charsets.UTF_8 else
Charsets.UTF_16
val langLen = flags and 0x3F
val lang = String(payload, 1, langLen, Charsets.US_ASCII)
val text = String(payload, langLen + 1, payload.size - langLen - 1,
textEncoding)
```

```
// Décodage RTD_URI → Android le fait automatiquement
val uri = record.toUri()
```

Lecture depuis le Manifest (app inactive) : ajouter `intent-filter` sur l'activité → `android:launchMode="singleTop"` pour éviter plusieurs instances.

Depuis Android/iOS (2018) : le système lit **automatiquement** les tags NDEF au déverrouillage de l'écran et propose d'ouvrir l'URL ou l'app correspondante — sans qu'une app dédiée soit active. Une URL NDEF sur un tag est traitée comme un lien universel.

Bluetooth Classique

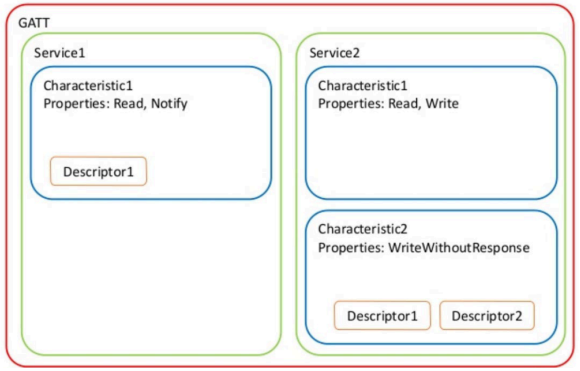
- Alternative sans-fil à USB, portée 10m, connexion bidirectionnelle chiffrée
- Peu adapté à la mobilité (consommation)
- Évolution : 1.0 (1999, 721 kbps) → 3.0 (2009) → 4.0 (2010, définit BLE)

Profils principaux

- **A2DP** : streaming audio (casque, enceinte)
- **HFP/HSP** : kit mains-libres
- **HID/HOGP** : clavier, souris, manette
- **PAN** : partage de connexion
- **PBAP** : accès au répertoire téléphonique (combiné HFP pour voiture)
- **AVRCP** : télécommande multimédia (TV, audio)
- **MAP** : accès aux messages
- Profil custom → **MFi obligatoire sur iOS**, possible mais complexe sur Android
- iOS supporte nativement : HFP 1.7, PBAP 1.2, A2DP 1.3, AVRCP 1.6, PAN, HID, MAP 1.1, Braille

Bluetooth Low Energy (BLE)

- Technologie **indépendante** du BT classique (non compatible)
- Portée 5-100m, débit utile 100 kbps, fonctionne des **mois sur pile bouton**
- Topologie **client/serveur**



Phases de communication

- **GAP** (avant connexion) : diffusion nom, services disponibles, connexion possible ?
- **GATT** (connecté) : services, characteristics, descripteurs

Structure GATT

- **Service** = ensemble de Characteristics (ex: Heart Rate 0x180D, Battery 0x180F, Current Time 0x1805)
  - Services standards → UUID 16 bits ; propriétaires → UUID 128 bits
- **Characteristic** = variable (max **20 bytes** par défaut = 1 paquet, **512 bytes** max)
  - Opérations : **Lecture**, **Écriture** (avec/sans confirmation), **Notification** (sans ACK) / **Indication** (avec ACK)
  - Le Central doit s'**inscrire** pour recevoir des notifications
- **Descriptor** = métadonnées d'une Characteristic

Exemple : Current Time Service 0x1805 (format des bytes de la Characteristic `0x2A2B`) :

Bytes	Contenu	Valeur ex.
0-1	Année (little-endian)	0xE6 0x07 = 2022
2	Mois (1-12)	0x03 = mars
3	Jour (1-31)	0x0F = 15
4	Heure (0-23)	0x0E = 14h
5	Minute (0-59)	0x1E = 30
6	Seconde (0-59)	0x00
7	Jour semaine (1=lundi)	0x02 = mardi
8	Fractions/256 s	0x00
9	Adjust reason	0x00

Sécurité / Appairage (Bonding)

- **Connecté** (non appairé) : échange en clair, tout appareil peut se connecter
- **Appairé/Bondé** : canal chiffré après échange de clés :
  - **Just Works** : automatique, vulnérable MITM
  - **Out of Band** : via NFC/Wi-Fi
  - **Passkey** : code PIN (écran + clavier)
  - **Numeric comparison** : 2 écrans, BLE 4.2+ uniquement

Évolutions BLE

- **5.0** (2016) : ×2 débit (2M PHY) ou ×4 portée (Coded PHY 200m), mesh networks
- **5.1** (2019) : calcul d'angle d'arrivée/départ (localisation en intérieur)
- **5.2** (2020) : profil audio BLE (LE Audio, remplace A2DP à terme)
- **5.3** (2021) : optimisations diverses
- **5.4** (2023) : chiffrement possible des annonces
- **6.0** (2024) : Channel Sounding (localisation précise)

Autres protocoles de proximité

- **Wi-Fi Direct** : connexion p2p Wi-Fi, idéal pour gros volumes
- **Google Nearby** : API p2p Android combinant BT + Wi-Fi + ultrasons
- **AirDrop** : Apple uniquement (macOS/iOS/iPadOS), BT + Wi-Fi, partage de fichiers

Synthèse & Bonnes Pratiques

Contraintes de Design Applicatif

Contrainte	Problème	Solution
Batterie	Émission = consommation	Regrouper requêtes, lazy poll, FCM
Latence	RTT élevé sur mobile	HTTP/3 QUIC, caching, look-ahead
Bande passante	Payload > capacité	Protobuf, DEFLATE, pagination
Connexion intermittente	Requête coupée = données perdues	Retry automatique, queue locale
Changement IP	Session TCP coupée	QUIC (connexion liée au token, pas IP)
Vie privée	Tracking FCM/NFC/BLE	Chiffrement E2E, permissions minimales
Multi-version	App ancienne + API nouvelle	Parseurs permissifs, versioning
Background	OS tue les apps	WorkManager, FCM, Foreground Service

Bonnes pratiques Android — Réseau & Performance

- **Ne jamais faire réseau/BDD sur le Main Thread** → ANR garanti → utiliser `Dispatchers.IO`
- **Désabonner les listeners** en `onPause()`/`onDestroy()` (LocationManager, BLE, NFC ForegroundDispatch)
- **Limiter les permissions** demandées au strict nécessaire (foreground avant background)
- **Gérer les null** : `lastLocation` peut être null si aucune position connue
- **Fermer les proxies CameraX** : `imageProxy.close()` à la fin de chaque image, sinon freeze
- **Timeout cache iBeacon** : retirer balises non vues depuis > 5 s, pas en temps réel (trop instable)
- **Geofencing rayon mini** : 100-150 m minimum ; délai de notification : 5 min pour économiser batterie

Checklist de choix de protocole

```
Données volumineuses, bande passante critique  → Protobuf + DEFLATE
Données simples, lisibilité nécessaire         → JSON
Services inter-entreprises, contrats forts     → SOAP (rarement sur
mobile)
Requêtes flexibles, app riche                  → GraphQL
Notifications serveur → client                 → FCM + poll déclenché
Échange p2p local gros volume                  → Wi-Fi Direct
Déclenchement à courte portée                  → NFC (< 4cm)
Suivi de présence par salle                    → iBeacon BLE
Positionnement précis < 30cm                   → UWB (AirTag /
Jetpack beta)
```